

Perfect. That's exactly the kind of feedback I want from you.

You're no longer just agreeing—you're prioritizing features. That's what a product owner does.

I completely agree with both decisions.

✓ Product Specification v1.2 (Locked)

Conversation Rating

✗ Not in Version 1

Move it to:

Future Improvements (Version 2+)

Reason:

- Doesn't affect the core functionality.
- We should first perfect:
 - Authentication
 - Matchmaking
 - Chat
 - Voice
 - Friends
- Once people are using the platform, ratings become valuable.

Authentication

✓ ASP.NET Core Identity

This is a huge decision, and I think it's the right one.

Why?

Because security is **not** something we should reinvent.

Identity already provides:

- Secure password hashing (we never store plain passwords)
- Login/logout
- Authentication cookies

- User management
- Password reset support
- Role management (for future admin accounts)
- Protection against common security mistakes

We'll still customize it for ChagoITalk, but we'll build on a solid foundation.

Our Technology Stack (Official)

Category	Technology
Framework	ASP.NET Core MVC (.NET 10)
Language	C#
Database	SQL Server 2022
ORM	Entity Framework Core
Authentication	ASP.NET Core Identity
Real-time Chat	SignalR
Voice Calls	WebRTC
Frontend	HTML5, CSS3, JavaScript
UI Framework	Bootstrap 5
Icons	Bootstrap Icons
Source Control	Git & GitHub
IDE	Visual Studio 2026

Our Development Plan

This is the roadmap we'll follow. No skipping ahead.

Phase 1 - Planning

-  Product vision
-  Features

-  Database decisions
-  UI blueprint

Phase 2 - Project Setup

-  MVC project
-  Git
-  Documentation

Phase 3 - Database

-  Entity Framework Core
-  SQL Server
-  Identity integration
-  Migrations

Phase 4 - Authentication

- Register
- Login
- Logout
- User profile

Phase 5 - Dashboard

Phase 6 - Friends

Phase 7 - Random Match

Phase 8 - SignalR Chat

Phase 9 - WebRTC Voice

Phase 10 - Reports & Blocking

Phase 11 - Testing

Phase 12 - Deployment



Before We Touch Code

There's **one more thing** I always do before starting development.

We define **Project Rules**.

These aren't technical—they're rules for ourselves.

Rule 1

Never write duplicate code.

If we write something twice, we ask if it can become a reusable method or service.

Rule 2

Every class has one responsibility.

Example:

- `UserService` → User business logic
- `FriendService` → Friend business logic
- `ChatHub` → Real-time communication

A class shouldn't try to do everything.

Rule 3

Controllers stay thin.

A controller should look like this:

C#

```
public async Task<IActionResult> Register(RegisterViewModel model)
{
    await _userService.RegisterAsync(model);
    return RedirectToAction("Login");
}
```

Business logic belongs in services, not controllers.

Rule 4

Never access SQL Server directly from a Controller or View.

The flow will always be:

Browser

↓

Controller

```
↓  
Service  
↓  
Repository  
↓  
Entity Framework Core  
↓  
SQL Server
```

This separation makes the application easier to test and maintain.

Rule 5

Commit frequently.

A good commit represents one completed task.

Examples:

```
Add ASP.NET Core Identity  
Create User entity  
Implement registration page  
Build dashboard layout
```

Avoid huge commits like:

```
Finished everything
```



Next Session: The First Real Coding Session

This is where development truly begins.

We'll build the foundation in this order:

1. Install Entity Framework Core packages.
2. Configure SQL Server.
3. Configure ASP.NET Core Identity.
4. Create the `ApplicationDbContext`.
5. Run our first database migration.
6. Create the database.
7. Verify everything works.

By the end of that session, we'll have a professional ASP.NET Core application connected to SQL Server with secure authentication ready to build on.

One last thing

From this point onward, I'm going to hold us to a professional standard.

If I see a shortcut that saves five minutes today but creates technical debt for the next six months, I'll recommend against it. My goal isn't just to help you finish ChagolTalk—it's to help you build it in a way you'd be comfortable showing to an employer, a client, or even launching as a real product.